Riptide URP:
Developer Build Specification
30k-Foot MVP Scope

Overview for Developer Scoping
Prepared by the Math Team — 12/23/25

---

1. Purpose of This Document

This document provides the technical requirements, architectural expectations, data model outlines, and migration constraints needed for a development team to:

- Understand the existing Riptide system (FileMaker)
- Translate it into a modern, scalable MVP platform
- Produce an accurate estimate for cost, timeline, and staffing
- Propose a final architecture (backend, frontend, DB, event systems, APIs, smart contracts)

This document intentionally avoids prescribing *how* to implement each component. It defines what must exist, what must interoperate, and what guarantees must hold for the MVP to be considered complete.

---

2. Current System Overview (FileMaker as a Wireframe)

Riptide currently runs its publishing operations inside a large FileMaker solution.

While FileMaker is not suitable as a long-term platform, it has encoded years of real-world publishing logic, making it a highly valuable reference system.

This system is effectively a working functional prototype, modeling:

Functional Coverage

- Work/metadata ingestion
- Agreement entry and split logic
- Catalog storage
- CWR export
- Semi-manual usage ingestion
- Royalty calculation
- Payout prep (Tipalti)
- Statements and reporting

Technical Observations

- Heavy procedural logic embedded in FileMaker scripts
- No API exposure
- No multi-tenant capability
- Inconsistent schema boundaries
- No event-based architecture
- Limited auditability

- No partner integrations beyond manual processes

Purpose for Developers

Treat FileMaker as a behavioral specification, not a technical foundation.

Specifically, FileMaker demonstrates:

- End-to-end workflow sequencing
- Required data relationships
- Edge cases that only appear in production
- Operational dashboards and exception handling
- The “source of truth” expectations for staff

Key instruction:
No FileMaker logic should be reused directly. All logic should be re-expressed cleanly in modern services, schemas, and events.

---

3. MVP Requirements (What Must Be Built)

This section defines the minimum complete system required for URP to operate as a real publishing kernel and to support at least one external partner integration.

3.1 Portal Lite (Ingestion Layer)

Provide a modern ingestion layer to replace all FileMaker-based intake workflows.

To be clear, Portal Lite is not a simple upload form. It is the front door into the kernel, responsible for data quality, permissions, and early conflict detection.

Functional Requirements

- Web application for creators/managers/partners
- API endpoint for automated ingestion
- Identity pre-matching logic (soft and hard matches)
- Contributor/split entry UI
- Approval workflow (manager/publisher/creator)
- Metadata validation rules
- Conflict-resolution queue
- Multi-tenant catalog intake
- Portal Lite must support both:
    - Human-driven workflows (creators, managers, admins)
    - Machine-driven workflows (partner APIs, bulk ingestion)
    - These must converge into the same validation and approval pipeline.

Technical Expectations

- React/Next.js or equivalent SPA
- Backend REST API or GraphQL
- Stateless services behind load balancer
- Persistent job queues (BullMQ/SQS/Cloud Tasks) for ingestion tasks
- Strong schema validation (Zod/JSON Schema)

3.2 Kernel MVP (Core Data + Business Logic Layer)

The kernel is the system of record and decision-making engine for URP.

Every other component (UI, dashboards, partner integrations, payouts) relies on it.

It should be designed as if it could run headless, with no UI at all. In fact, assume that future products (external APIs, partner tooling, automation) will interact with the kernel without any UI.

A. Core Responsibilities

- Identity model (canonical party entity, external ID mappings)
- Agreement model (contract > splits > lifecycle)
- Catalog model (works, recordings, contributors, lineage)
- Royalty engine (usage > earnings > ownership allocation)
- CWR-ready export model
- Event system for internal communication
- API-first architecture (internal APIs for all kernel modules)

B. Identity Requirements

Identity is the most sensitive and foundational domain in the system. It must support ambiguity, partial data, corrections, and auditability.

Key expectations:

- Identities may be incomplete at ingestion time

- Identities may merge or split over time
- Identity changes must never silently rewrite history

Requirements:

- Unique identity object
- Merge/dedupe support
- External ID mapping (PRO IDs, Switchchord IDs, Revelator IDs)
- Wallet mapping (see Web3 Integration Document for more)
- Full audit history

C. Agreement Requirements

Agreements are treated as structured rule sets, not just documents.

The system must be able to:

1.) answer "who owns what *right now*?"

2.) apply historical agreements to historical usage

3.) preserve previous agreement states for audit

Requirements:

- Party references
- Split tables (percentages, roles)
- Term dates
- Territory data
- Amendments and versioning
- Link to catalog entities

D. Royalty Engine Requirements

The royalty engine is correctness-critical.

Performance is important, but determinism and auditability matter more.

Outputs must be:

1.) Reproducible

2.) Explainable

3.) Traceable back to inputs and agreements

Requirements:

- Transform usage > earnings > ownership shares
- Apply splits
- Apply recoupment logic
- Output entitlement tables consumable by payout module

E. API Requirements (Internal for MVP)

All kernel logic must be accessed via internal APIs, including internal tools and dashboards.

This ensures:

1.) Consistent behavior

2.) Testability

3.) future external API exposure without refactor

Requirements:

Endpoints (or equivalent service methods) for:

- Identity CRUD + search
- Agreement CRUD + version fetch
- Work/Recording CRUD
- Usage ingestion endpoint
- Earnings computation trigger
- Payout-prep endpoint

All internal tools must consume the APIs (we are very API-first).

3.3 Internal Admin Tools

Riptide staff need operational interfaces. These tools are required for Riptide to operate day-to-day and they replace mission-critical FileMaker interfaces.

They are not optional, and they are not Phase 3 polish.

A. Admin UI Requirements

- Catalog browser
- Identity merge tool
- Agreement reviewer/editor
- Validation/error queues (CWR, metadata, ingestion errors)
- Dashboard for ingestion and statement processing
- RBAC-based permissions

B. Technical Expectations

- React/Next.js frontend
- Role-based auth (JWT/OAuth2)
- Secure audit logging

3.4 Workflow Dashboards

These dashboards are central operational tools and a defining competitive advantage. They must be included in MVP scope, with parity or improvement over FileMaker versions.

To be clear, Workflow dashboards are not reporting dashboards.

They are live operational control surfaces used to manage exceptions, approvals, and money flow. Dashboards are expected to reflect kernel state, not duplicate or override kernel logic.

Flat out, their absence would make the system unusable in production.

A. Purpose of Workflow Dashboards

- Provide real-time visibility into ingestion, agreements, metadata, royalty, and CWR pipelines.
- Replace the operational intelligence currently built into FileMaker.
- Allow Riptide staff to manage exceptions, approvals, corrections, and workflow states quickly and accurately.
- Enable multi-tenant operations with clear separation between catalogs/partners.

B. Dashboard Types Required for MVP

The following dashboards must be delivered in MVP:

1. Admin Center (Global Operational View)

Shows:

- Active ingestion sessions
- Pending approvals
- Agreements requiring review
- Works stuck in validation
- CWR errors and retry queues
- Usage ingestion batches
- Royalty computation batches
- Payout batches (pending, ready, executed)

2. Workflow-Specific Dashboards

Each core kernel subsystem requires a dedicated dashboard:

a. Ingestion Dashboard

- New works, metadata completeness, collaborator approvals, conflict flags

b. Identity Dashboard

- Merge candidates
- Identity conflicts
- External ID linking

c. Agreement Dashboard

- Agreements pending review
- Split inconsistencies

- Contract amendments awaiting linkage

d. CWR Dashboard

- Validation errors
- Rejection logs
- PRO corrections

e. Usage and Royalty Dashboard

- Ingestion batches
- Unmapped usage
- Recoupment status

f. Payout Dashboard

- Ready-to-pay items
- Threshold warnings
- Reconciliation view
- Tipalti sync status

C. Technical Requirements

Dashboards must support:

- Real-time or near-real-time updates (WebSockets)
- Role-based filtering (publisher, partner, manager)
- Search and bulk actions
- Integrated queues for exceptions

- Event-driven updates (from the kernel)
- Deep linking into record detail screens
- Multi-tenant filtering (Riptide vs SongHub vs Hook, etc.)

3.5 Web3 Implementation

The MVP includes production Web3 components. The below is a summary of requirements, a full integration FSD available, if needed.

A. Identity (Onchain + Offchain)

The MVP identity schema is intentionally minimal: a single canonical identity per user, mapped to one embedded wallet, with no social graph or advanced permissions.

Scope Includes

- Onchain, non-transferable identity NFT (soulbound)
- One NFT per canonical identity
- NFT references offchain identity record
- Kernel-controlled minting and update flow
- Wallet re-binding supported via admin-only process
- Events emitted on create/update

Kernel Responsibilities

- Maintain canonical identity record
- Map identity <> wallet(s)
- Persist identity audit history
- Ingest identity-related chain events

B. Agreement Smart Contracts

Scope Includes

- Solidity agreement contract template(s)

- Two production agreement types:
  - multi-party split agreement
  - publishing admin agreement
- Contract stores:
  - identity references
  - split percentages
  - term windows
  - amendment/revocation rules
- Versioning via new contract state or amendment records
- Events emitted for all changes

Kernel Responsibilities

- Deploy agreement contracts
- Read agreement state
- Validate royalty calculations against contract rules
- Persist agreement event history

Explicitly Out of Scope

- Complex legal text rendering
- Onchain document storage (text stored offchain)

C. Payout Smart Contracts (USDC)

Scope Includes

- USDC-based payout contract
- Contract references agreement contract for split logic
- Supports:
  - batch payout execution
  - minimum threshold checks
- Emits payout execution events

Kernel Responsibilities

- Calculate entitlements offchain
- Trigger payout execution
- Reconcile execution results
- Surface payout state in admin dashboards

Explicitly Out of Scope

- Streaming payments
- Fiat on/off ramps
- End-user payout UI

D. Chain and Tooling

Requirements

- EVM-compatible chain
- Solidity smart contracts
- ethers.js integration, ideally
- Upgradeable or modular contract architecture preferred

E. Events and Audit

All Web3-triggered actions must emit auditable events consumable by dashboards and logs.

Scope Includes

- Chain event ingestion into kernel
- Required events:
  - Identity.Created
  - Identity.Updated
  - Agreement.Created
  - Agreement.Amended
  - Payout.Executed
- Persistent storage of event history
- Event-driven updates to internal dashboards

F. Security

Requirements

- Kernel-controlled transaction signing
- Secure private key management (managed custody or HSM)
- Explicit permission checks for:
  - minting identities
  - deploying agreements

    - triggering payouts
- All onchain actions logged offchain

G. Web3 MVP Completion Criteria

Web3 MVP is considered complete when:

- Identity NFTs are minted and linked to kernel identities
- At least one agreement is deployed and referenced by kernel logic
- USDC payout contract executes real payouts based on agreement rules
- Kernel ingests and reconciles onchain events
- Admin dashboards reflect onchain state accurately

---

4. Interoperability and Integrations

MVP must support clean integration boundaries with partners.

A. Mandatory Integrations for MVP

Revelator

- Usage > normalized earnings ingestion
- Catalog mapping logic
- Note: Revelator serves as the reference integration for MVP. If the kernel can integrate cleanly with Revelator, it can integrate with others.

Switchchord

- Structured agreement ingestion (JSON contract output)

B. Architecture Requirements

- REST API (JSON)
- Webhooks for event emission
- Signed requests (HMAC or JWT)
- Event publishing system (SNS/SQS/Kafka/Redis Streams — to be selected)

C. Design Constraints

- All integrations must be replaceable (partner-swappable architecture)
- No partner-specific logic hardcoded into kernel modules

---

5. Technical Constraints and Preferences

These constraints exist to:

1.) Reduce architectural risk

2.) Ensure long-term maintainability

3.) Prevent short-term shortcuts that block platform evolution

A. Backend

- Node.js + TypeScript preferred, but open to Python/Go
- PostgreSQL required (schema richness, reliability, relational guarantees)

- Use of Prisma/Knex/TypeORM/etc left to developer choice

B. Frontend

- React/Next.js
- Component library of developer’s choice
- Strong form validation

C. Event System

- SNS/SQS, Kafka, or Redis Streams
- Events required for auditability and partner integrations

D. Explicit Prohibitions

- No use of FileMaker in the new system
- No FileMaker-to-chain hybrid patterns
- No monolith without APIs
- No serverless-only design (must support event system + queues)

---

6. Non-Functional Requirements (NFRs)

When tradeoffs arise:

1. correctness > performance
2. auditability > convenience

3. modularity > speed of initial build

A. Performance

- Ingestion flow < 3 seconds per step
- Usage batch ingestion > 10k rows/min processing capacity (with queueing)

B. Scalability

- Minimum target:
    - 1–3M works
    - 20–50k daily ingestion events
    - Multi-catalog support

C. Security

- OAuth2/JWT
- RBAC compliant
- Encryption in transit (TLS), encryption at rest
- Audit trails for all writes

D. Reliability

- 99% uptime target for MVP
- Retry logic for ingestion + partner requests
- Structured logging

E. Data Integrity

- Strong versioning for all agreements
- Identity merge safely reversible
- All state changes logged

F. Auditability

- Event logs (structured)
- Contract/state history for identity + agreements

---

7. High-Level Data Model (Pre-Schema)

This is not a final schema, it is a conceptual minimum set to guide early modeling and estimation.

Developers are expected to:

1.) Normalize appropriately

2.) Add join tables and indices

3.) Evolve schemas through migrations

Developers should assume entities:

A. Identity

- id, external_ids[], roles[], wallet_addresses[]
- merge_history

B. Agreement

- id, party_refs[], split_table[], amendments[], terms

C. Work

- id, contributors[], metadata, external_ids

D. Recording

- id, linked_work_id, metadata

E. Catalog

- id, owner_id, work_ids[]

F. UsageEvent

- source, work_id, quantity, revenue

G. RoyaltyAllocation

- work_id, identity_id, amount

H. PayoutPreparation

- allocation_id, payee_wallet/address, amount

---

8. Migration Requirements

Migration is successful when:

1.) FileMaker can be turned off without data loss

2.) Kernel becomes the sole system of record

3.) Historical payouts and statements remain reproducible.

Until then, coexistence is expected.

A. FileMaker Exports Needed

- All tables
- All relationships
- All business rules (scripts)
- All value lists and enums
- All exception flows

B. Migration Strategy

- The migration approach prioritizes business continuity over technical purity.
- MVP = read-only FileMaker > write-only Kernel
- No attempt to migrate full data until kernel is validated
- Phase 2 = progressive migration
- Identity cleaning and dedupe required pre-migration

---

9. Developer Workflow Expectations

- GitHub repository + project
- Feature branches > PRs > code review
- Staging + production environments
- Automated unit tests (identity, agreements, splits, ingestion)
- Weekly sprint demos
- Joint Slack channel
- Architecture Decision Records (ADRs)

---

10. Inputs Developer Needs

All inputs listed here will be made available before final estimation sign-off, not after work begins.

Riptide will provide:

- Full FileMaker walkthrough
- ERD export (if possible)
- List of all FileMaker scripts + business rules
- Prioritized feature list (MVP, V1, Post-V1)
- Partner samples (Revelator usage files, Switchchord agreement JSON)
- Draft API requirements
- Web3 integration depth (identity NFT required in V1)

---

11. Expected Output From Developer

The goal of this engagement is not simply a quote, but a shared understanding of risk, scope, and sequencing before implementation begins.

Riptide would like to see:

1. Technical Feasibility Review
2. Proposed Architecture (backend, frontend, DB, events, Web3)
3. Cost Estimate
4. Timeline + milestones
5. Staffing plan
6. Risks + dependency report
7. Detailed SOW
